Secure Software Design and Development

Security Test Report

Submitted to:

Engr Muhammad Ahmad Nawaz

Prepared by:

Maheen Aamir (SP23-BCT-027)

Malik Muhammad Huzaifa (SP23-BCT-029)

Muzamil Ali Akhtar (SP23-BCT-043)

Maryum Masood (SP23-BCT-031)

Department of computer science

(Cyber Security)

Scope

This report summarizes the security verification performed on the Flask To-Do app after implementing key mitigations:

- **IDOR** fix in task authorization (app/routes/tasks.py)
- **CSRF** protection via Flask-WTF (app/__init__.py + form tokens)
- **Clickjacking** prevention using HTTP headers (X-Frame-Options, CSP frame-ancestors)

CI/CD reference: .github/workflows/ci-cd.yml

Relevant jobs: codeql, zap-baseline, fail-on-critical

1. Security Testing Methodology

1.1 SAST (Static Application Security Testing)

- Tool: **CodeQL** via GitHub Actions codeql job
- Scope: Analyze Python source code, Flask routes, and templates for security anti-patterns
- Checks performed:
    - Input validation and sanitization
    - Authorization enforcement patterns
    - Sensitive data exposure (passwords, secrets)
    - CSRF protection on forms
    - Clickjacking headers

1.2 DAST (Dynamic Application Security Testing)

- Tool: **OWASP ZAP Baseline** via Docker in CI
- Scope: Scan running Flask app (http://127.0.0.1:5000) for runtime security issues
- Checks performed:
    - Missing or misconfigured security headers
    - Form CSRF tokens and validation
    - Session cookie security
    - Basic XSS and injection patterns
    - Automated request replay for state-changing endpoints

1.3 Manual Verification

- IDOR: tested by attempting cross-user task access in multi-user scenarios
- CSRF: tested by removing/altering csrf_token from forms and confirming rejection
- Clickjacking: verified by attempting to load the app in an iframe

2. Test Observations and Evidence

2.1 IDOR

- Before fix: Any authenticated user could manipulate task IDs in URL (/tasks/<id>/edit) to access or delete another user's task.
- After fix: _get_user_task_or_404() ensures queries include current_user.id; unauthorized access returns **404**.
- Evidence: manual tests confirmed cross-user access is blocked.

2.2 CSRF

- Before fix: POST requests (task delete, logout) could be executed by malicious pages without user consent.
- After fix: Global CSRFProtect enabled, all forms contain hidden csrf_token. Requests missing/altered token are rejected.
- Evidence: ZAP baseline scan shows no CSRF alerts, functional tests reject invalid tokens.

2.3 Clickjacking

- Before fix: App could be embedded in third-party frames, allowing UI redress attacks.
- After fix: HTTP headers added:
  - X-Frame-Options: DENY
  - Content-Security-Policy: frame-ancestors 'none'
- Evidence: Attempted iframe embedding blocked in browser; ZAP confirms headers present.

2.4 Session & Cookie Security

- SESSION_COOKIE_SAMESITE = "Lax"
- SESSION_COOKIE_SECURE = True (recommended for HTTPS deployment)
- Evidence: Browser inspection confirms correct cookie attributes.

2.5 Data Leakage Prevention

- Debug mode disabled (DEBUG = False)
- Error messages sanitized
- Evidence: Stack traces and internal paths not exposed on invalid requests.

3. Vulnerability Summary

Vulnerability	Before Fix	After Fix	Validation Method
IDOR	Cross-user task access possible	_get_user_task_or_404 enforces ownership	Manual authz test + code review + SAST
CSRF	Forms vulnerable	CSRFProtect + csrf_token in all forms	Functional test + DAST + SAST
Clickjacking	Pages embeddable	HTTP headers (DENY, CSP)	DAST header checks + browser verification
Data Leakage	Debug info visible	Debug disabled, errors sanitized	Manual test + review of error responses
Session Hijacking	Cookies not restricted	SAMESITE=Lax, secure cookie recommended	Browser inspection

4.CI/CD Pipeline Verification

- **.github/workflows/ci-cd.yml ensures:**
  - Unit tests pass (unittest)
  - CodeQL static analysis (security-extended)
  - ZAP baseline dynamic scan
  - fail-on-critical job fails build if any critical CodeQL alert exists

- **Evidence collected as artifacts:**
  - zap-baseline-report.html, .md, .json
  - Flask app logs (flask.log)
- **Benefits:**
  - Automated prevention of introducing regressions or new vulnerabilities
  - Ensures consistent security checks on every push or pull request

5. Recommendations and Future Work

1. **Automated Authorization Tests**
   - Implement tests to verify IDOR protection in multi-user scenarios.
2. **Advanced DAST Scans**
   - Integrate full ZAP spider and authenticated scans for stronger assurance.
3. **Secure Deployment**
   - Enforce HTTPS and production-grade secrets management.
   - Monitor CodeQL and ZAP alerts for each PR to catch regressions.
4. **Logging and Auditing**
   - Add audit logs for task operations and login/logout actions.
5. **Continuous Security Improvement**
   - Regularly update dependencies and scan for new CVEs.

6. Conclusion

After implementing the IDOR, CSRF, and Clickjacking fixes, the Flask To-Do app is resilient against these common web vulnerabilities. The CI/CD pipeline enforces automated verification with SAST and DAST tools, ensuring security controls are maintained throughout development.
Residual risk is minimal but can be further reduced through automated authorization tests and advanced DAST scans.